PROJECT REPORT OF INDUSTRY ORIENTED HANDS ON EXPERIENCE (IOHE)

ON

Machine Learning Enabled Intrusion Detection for Intelligent Network Security

submitted in partial fulfilment of the requirements for the award of degree of

BACHELOR OF ENGINEERING

In

COMPUTER SCIENCE AND ENGINEERING

Submitted by:

Saniya Ahuja(2210990787)

Mehak(2210990571)

Supervised By:

Dr. Monika Aggarwal

Assistant Professor

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
CHITKARA UNIVERSITY INSTITUTE OF ENGINEERING AND TECHNOLOGY
CHITKARA UNIVERSITY, PUNJAB, INDIA

CONTENTS

S no.	Title	Page No.
	Acknowledgment	4
	Abstract	5
1	Introduction	6-8
2	Methodology	8-14
3	Tools and Technologies	14-20
4	Implementation	20-24
5	Results	25-28
6	Conclusion and Future Scope	28-29
7	References	29-30
8	Appendices	31-34

DECLARATION

I hereby certify that the work which is being presented in the project report entitled **"**Machine Learning Enabled Intrusion Detection for Intelligent Network Security**"** in partial fulfilment of requirement for the award of the degree of Bachelor of Engineering (Computer Science and Engineering) submitted in the department of Computer Science and Engineering at Chitkara University Institute of Engineering and Technology, Chitkara University, Punjab, India, is an authentic record of my own work carried out under the supervision of **Dr. Monika Aggarwal** . The matter presented in this project report has not been submitted in any other university/institute for the award of any degree.

Place: Rajpura **(Saniya Ahuja , Mehak)**

Date: …….. (2210990787,2210571)

This is to certify that the above statement made by the candidate is correct to the best of my knowledge and belief.

Dr Monika Aggarwal

Assistant Professor

Department of Computer Science and Engineering

Chitkara University Institute of Engineering and Technology,

Chitkara University, Punjab, India

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my mentor, Dr. Monika Aggarwal, Department of Computer Science and Engineering, Chitkara University Institute of Engineering and Technology, for her valuable guidance, continuous support, and encouragement throughout the completion of this project titled "Machine Learning Enabled Intrusion Detection for Intelligent Network Security."

I am highly thankful to the faculty members of the Department of Computer Science and Engineering for providing the necessary resources and academic environment that helped me successfully complete this work.

I would also like to thank my friends and classmates for their cooperation, suggestions, and motivation during the development of the project.

Finally, I express my heartfelt gratitude to my parents and family members for their constant encouragement and support throughout my academic journey.

Abstract

With the rapid growth of internet usage and digital communication, network security has become a major concern for organizations and individuals. Traditional intrusion detection systems are often unable to detect modern and evolving cyberattacks effectively because they mainly depend on predefined attack signatures. To overcome these limitations, Machine Learning based Intrusion Detection Systems (IDS) have gained significant attention due to their ability to learn patterns from network traffic and identify malicious activities automatically.

This project presents a Machine Learning Enabled Intrusion Detection System for intelligent network security using supervised learning techniques. The system is designed to analyze network traffic data and classify it as normal or malicious. The NSL-KDD dataset has been used for training and testing the models because it contains different categories of network attacks and normal traffic records. Data preprocessing techniques such as label transformation, one-hot encoding, feature scaling, and dataset splitting were applied to improve model performance and accuracy.

Three machine learning algorithms namely Logistic Regression, Support Vector Machine (SVM), and Random Forest were implemented and compared in this project. The performance of the models was evaluated using standard classification metrics including accuracy, precision, recall, F1-score, confusion matrix, and ROC-AUC score. Experimental results showed that the Random Forest classifier achieved the highest detection accuracy and provided better performance compared to the other models.

The proposed intrusion detection system demonstrates that machine learning techniques can significantly improve network security by accurately detecting malicious activities while reducing false alarms. The system can be further extended for real-time intrusion detection and advanced cybersecurity applications

1. Introduction

1.1 Background of Cybersecurity and Networks

In today's digital world, computer networks are widely used in banking systems, educational institutions, healthcare organizations, industries, cloud computing, and government sectors. Every organization depends heavily on internet communication and data sharing for daily operations. As the use of online services increases, the amount of sensitive and confidential information transferred through networks also increases.

However, the rapid growth of internet technology has also increased the number of cyberattacks. Hackers and attackers continuously attempt to gain unauthorized access to systems, steal private data, interrupt network services, or damage digital infrastructure. Common cyber threats include malware attacks, phishing attacks, ransomware, denial-of-service attacks, data breaches, and unauthorized access.

Because of these security threats, protecting computer networks has become one of the most important tasks in cybersecurity. Organizations require intelligent systems that can monitor network traffic continuously and detect suspicious activities before they cause damage.

1.2 Need for Intrusion Detection Systems (IDS)

An Intrusion Detection System (IDS) is a security mechanism that monitors network traffic and identifies malicious activities or policy violations. The main purpose of IDS is to detect attacks and generate alerts whenever suspicious behavior is identified in the network.

IDS plays a significant role in modern cybersecurity because firewalls alone cannot completely protect networks from internal attacks or advanced cyber threats. IDS helps administrators monitor traffic patterns and respond quickly to security incidents.

There are mainly two types of intrusion detection systems:

- **Signature-Based IDS**

This system detects attacks by comparing network traffic with already known attack signatures or patterns. It works effectively for previously identified attacks but cannot detect new or unknown attacks.

- **Anomaly-Based IDS**

This system learns normal network behavior and identifies deviations from standard patterns. If unusual activity occurs, it is considered suspicious. This approach is more suitable for detecting zero-day attacks and unknown threats.

1.3 Limitations of Traditional Security Systems

Traditional intrusion detection systems mainly depend on rule-based and signature-based techniques. Although these systems are useful for detecting known attacks, they face several limitations in modern network environments.

Some major limitations are:

- Inability to detect unknown or zero-day attacks
- High dependency on manually updated signatures
- Poor adaptability to changing attack patterns
- Difficulty in handling large volumes of network traffic
- Increased false positive and false negative rates

Modern cyberattacks evolve continuously, making traditional systems less effective. Therefore, there is a need for intelligent and adaptive security solutions that can automatically learn attack behaviour.

1.4 Role of Machine Learning in Intrusion Detection

Machine Learning (ML) is a branch of Artificial Intelligence that enables systems to learn from historical data and make predictions without being explicitly programmed. ML algorithms can analyze large amounts of network traffic data and identify hidden patterns associated with malicious activities.

Machine learning improves intrusion detection systems in several ways:

- Automatically learns network behaviour
- Detects unknown and evolving cyber threats
- Improves detection accuracy
- Reduces manual effort
- Handles large-scale network traffic efficiently
- Enhances real-time monitoring capabilities

ML-based IDS systems can continuously improve their performance through training and data analysis. Because of these advantages, machine learning has become one of the most important technologies in cybersecurity research.

2. Methodology

The proposed project implements a Machine Learning based Intrusion Detection System (IDS) for identifying malicious activities in network traffic. The methodology follows a structured process that includes data collection, preprocessing, feature engineering, model training, intrusion detection, and performance evaluation. The complete workflow is designed to improve the accuracy and reliability of network security systems by automatically identifying suspicious behavior in network communication.

The methodology consists of the following major stages:

1. Data Collection
2. Data Preprocessing
3. Feature Extraction and Selection
4. Machine Learning Model Training
5. Intrusion Detection and Prediction
6. Performance Evaluation and Visualization

2.1 Data Collection

The first step in the methodology is collecting an appropriate dataset for training and testing the intrusion detection system. In this project, the **NSL-KDD dataset** is used because it is one of the most widely accepted benchmark datasets for intrusion detection research.

The NSL-KDD dataset contains network traffic records categorized into normal and attack classes. It includes different types of attacks such as:

- Denial of Service (DoS)
- Probe Attacks
- User to Root (U2R)
- Remote to Local (R2L)

Dataset Characteristics

- Total Records: 22,544
- Number of Features: 41

- Target Variable: Normal or Attack
- Data Type: Network Traffic Records

Each record in the dataset represents a network connection and contains several attributes such as protocol type, service type, duration, source bytes, destination bytes, and connection flags.

2.2 Data Preprocessing

Raw network traffic data cannot be directly used for machine learning because it contains categorical values, redundant information, and features with different scales. Therefore, preprocessing is necessary to improve model performance and accuracy.

2.2.1 Label Transformation

The target labels in the dataset are converted into binary classes:

- Normal Traffic → 0
- Attack Traffic → 1

This transformation simplifies the classification problem and enables binary prediction.

2.2.2 Handling Categorical Features

Some features in the dataset are categorical in nature, such as:

- Protocol Type
- Service
- Flag

Machine learning algorithms require numerical inputs; therefore, these categorical attributes are converted into numerical format using **One-Hot Encoding**.

One-Hot Encoding creates separate binary columns for each category, ensuring that no numerical relationship is assumed between categories.

2.2.3 Feature Scaling

The numerical features in the dataset have different ranges. To ensure fair contribution of all features during model training, feature scaling is applied using **Standard Scaling**.

Feature scaling normalizes the data by transforming it into a standard distribution with:

- Mean = 0
- Standard Deviation = 1

This improves model convergence and stability.

2.2.4 Dataset Splitting

The preprocessed dataset is divided into training and testing datasets.

- Training Data → 80%
- Testing Data → 20%

The training dataset is used for learning network behaviour patterns, while the testing dataset evaluates the model's performance on unseen data.

2.3 Feature Extraction and Selection

Feature extraction is an important step because not all network attributes contribute equally to intrusion detection.

Relevant features such as:

- Source Bytes
- Destination Bytes
- Connection Duration
- Flag Status
- Service Type

are analyzed to determine their impact on identifying malicious activity.

Feature importance analysis is later performed using the Random Forest classifier to identify the most significant attributes affecting intrusion detection performance.

2.4 Machine Learning Model Training

The project uses supervised machine learning algorithms to classify network traffic into normal or malicious categories.

Three machine learning models are implemented and evaluated:

2.4.1 Logistic Regression

Logistic Regression is a statistical classification algorithm used for binary classification problems.

The model predicts the probability of a network connection being malicious or normal based on input feature values.

Advantages:

- Simple and efficient
- Fast training process
- Easy interpretation

2.4.2 Support Vector Machine (SVM)

Support Vector Machine is a powerful classification algorithm that separates data using an optimal hyperplane.

Linear SVM is used in this project because it performs effectively with high-dimensional network traffic data.

Advantages:

- High classification capability
- Effective in complex datasets
- Good generalization performance

2.4.3 Random Forest

Random Forest is an ensemble learning algorithm that combines multiple decision trees to improve classification performance.

It works by generating several trees during training and selecting the final prediction through majority voting.

Advantages:

- High accuracy
- Reduced overfitting
- Better handling of complex data patterns

Random Forest achieved the best performance among all evaluated models.

2.5 Intrusion Detection Process

Once the machine learning models are trained, the system can analyze incoming network traffic and classify it as normal or malicious.

The intrusion detection process includes:

1. Capturing network traffic
2. Preprocessing incoming data

3. Extracting important features
4. Passing data to the trained model
5. Predicting attack or normal traffic
6. Generating alerts for suspicious activity

The IDS continuously monitors network activity in real time to ensure network security.

2.6 Performance Evaluation

The performance of the proposed Intrusion Detection System (IDS) is evaluated using various classification metrics. These metrics help in measuring how effectively the machine learning models identify malicious and normal network traffic.

The evaluation is based on the values obtained from the **Confusion Matrix**, which contains four important components:

- True Positive (TP)
- True Negative (TN)
- False Positive (FP)
- False Negative (FN)

2.6.1 Accuracy

Accuracy measures the percentage of correctly classified records.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

Where:

- TP = True Positive
- TN = True Negative
- FP = False Positive
- FN = False Negative

2.6.2 Precision

Precision measures how many predicted attacks are actually attacks.

$$Precision = \frac{TP}{TP + FP}$$

2.6.3 Recall

Recall measures the ability of the model to correctly identify malicious traffic.

$$Recall = \frac{TP}{TP + FN}$$

2.6.4 F1-Score

F1-Score provides a balance between precision and recall.

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

2.6.5 Confusion Matrix

A Confusion Matrix is a performance evaluation table used to compare the actual class labels with the predicted class labels generated by the machine learning model.

It provides detailed information about the classification performance of the intrusion detection system.

	Predicted Normal	**Predicted Attack**
Actual Normal	True Negative (TN)	False Positive (FP)
Actual Attack	False Negative (FN)	True Positive (TP)

The confusion matrix is mathematically represented as:

$$Confusion\ Matrix = \begin{bmatrix} TN & FP \\ FN & TP \end{bmatrix}$$

True Positive (TP)

True Positive represents the number of malicious network activities that are correctly identified as attacks by the intrusion detection system.

Example:

If a cyberattack occurs and the IDS correctly detects it as malicious traffic, it is counted as a True Positive.

➞ Actual Attack + Predicted Attack = True Positive

True Negative (TN)

True Negative represents the number of normal network activities that are correctly classified as normal traffic.

Example:

If a genuine user accesses the network and the IDS correctly identifies it as safe traffic, it is counted as a True Negative.

➞ Actual Normal + Predicted Normal = True Negative

False Positive (FP)

False Positive occurs when the intrusion detection system incorrectly classifies normal traffic as malicious activity.

Example:

If a normal user performs legitimate activity but the IDS marks it as an attack, it becomes a False Positive.

➞ Actual Normal + Predicted Attack = False Positive

False Positives create unnecessary alerts and increase system overhead.

False Negative (FN)

False Negative occurs when malicious traffic is incorrectly classified as normal traffic.

Example:

If a hacker attacks the network but the IDS fails to detect the attack and considers it safe traffic, it is counted as a False Negative.

➞ Actual Attack + Predicted Normal = False Negative

False Negatives are dangerous because real attacks remain undetected.

2.7 Visualization and Analysis

The project also includes graphical analysis to better understand model performance.

The following visualizations are generated:

- Confusion Matrix
- ROC Curve
- Feature Importance Graph
- Model Comparison Charts

These visualizations help in analyzing detection capability and identifying the best performing machine learning model.

3. Tools and Technologies

The proposed intrusion detection system was developed using various tools, technologies, programming libraries, and machine learning frameworks. These technologies helped in data

preprocessing, model training, visualization, and performance evaluation of the intrusion detection models.

3.1 Python Programming Language

Python was used as the primary programming language for the implementation of the intrusion detection system. Python is widely used in machine learning and cybersecurity applications because of its simple syntax, large collection of libraries, and strong community support.

Advantages of Python:

- Easy to understand and implement
- Supports machine learning and data science libraries
- Efficient handling of large datasets
- Provides strong visualization support

Python was used for:

- Data preprocessing
- Feature engineering
- Model training
- Performance evaluation
- Visualization of results

3.2 Jupyter Notebook

Jupyter Notebook was used as the development environment for coding and experimentation. It provides an interactive interface where code, output, graphs, and explanations can be executed in a single document.

Features:

- Interactive coding environment
- Easy debugging and testing
- Supports visualization and documentation
- Helpful for machine learning experiments

Jupyter Notebook helped in:

- Running machine learning models
- Visualizing graphs and confusion matrices
- Comparing model performances

3.3 Pandas Library

Pandas is a Python library used for data analysis and manipulation. It provides powerful data structures such as DataFrames for handling structured datasets.

Uses in the Project:

- Loading the NSL-KDD dataset
- Cleaning and organizing data
- Handling missing values
- Data transformation and preprocessing

Important Functions Used:

- read_csv()
- DataFrame()
- drop()
- replace()

3.4 NumPy Library

NumPy is a numerical computing library used for performing mathematical and array operations efficiently.

Uses in the Project:

- Numerical computations
- Array manipulation
- Mathematical transformations
- Feature normalization

Advantages:

- Faster computation
- Efficient memory usage
- Supports multidimensional arrays

3.5 Scikit-learn Library

Scikit-learn is one of the most important machine learning libraries used in the project. It provides various algorithms and tools for classification, preprocessing, model training, and evaluation.

Machine Learning Algorithms Used:

- Logistic Regression

- Linear Support Vector Machine (SVM)
- Random Forest Classifier

Scikit-learn Modules Used:

- train_test_split
- StandardScaler
- OneHotEncoder
- LogisticRegression
- SVC
- RandomForestClassifier
- confusion_matrix
- classification_report
- roc_curve
- accuracy_score

Purpose:

- Training machine learning models
- Evaluating classification performance
- Generating confusion matrices and ROC curves

3.6 Matplotlib

Matplotlib is a data visualization library used for plotting graphs and charts.

Uses in the Project:

- Plotting performance comparison graphs
- Drawing ROC curves
- Visualizing feature importance
- Displaying confusion matrices

Advantages:

- High-quality visualizations
- Easy customization
- Supports multiple graph formats

3.7 Seaborn

Seaborn is a statistical visualization library built on top of Matplotlib. It was used to create attractive and informative graphical representations.

Uses:

- Heatmap visualization of confusion matrices
- Statistical graph plotting
- Comparative model analysis

Advantages:

- Better visualization aesthetics
- Easy integration with Pandas DataFrames
- Simplifies complex plotting

3.8 NSL-KDD Dataset

The NSL-KDD dataset was used for training and testing the intrusion detection system. It is a benchmark dataset widely used in network intrusion detection research.

Dataset Characteristics:

- 22,544 records
- 41 network traffic features
- Binary classification:
    - Normal Traffic
    - Malicious Traffic

Attack Categories:

- Denial of Service (DoS)
- Probe Attacks
- User to Root (U2R)
- Remote to Local (R2L)

Purpose:

- Training machine learning models
- Evaluating intrusion detection performance

3.9 Machine Learning Techniques

The project used supervised machine learning algorithms for intrusion detection.

(a) Logistic Regression

A statistical classification algorithm used for binary prediction problems.

Advantages:

- Simple and fast
- Effective for linear classification
- Low computational cost

(b) Support Vector Machine (SVM)

SVM is a classification algorithm that separates classes using an optimal hyperplane.

Advantages:

- Works well with high-dimensional data
- Effective for network traffic classification
- Good generalization capability

(c) Random Forest

Random Forest is an ensemble learning algorithm that combines multiple decision trees.

Advantages:

- High accuracy
- Reduces overfitting
- Handles complex datasets effectively

The Random Forest model achieved the best results in the project with:

- Accuracy: 99.26%
- Precision: 99.24%
- Recall: 99.29%
- F1-Score: 99.27%

3.10 Operating System

The project was developed and executed on a Windows operating system environment.

System Requirements:

- Windows 10/11
- Python 3.x
- Jupyter Notebook
- Minimum 8GB RAM

3.11 Hardware Requirements

The following hardware specifications were sufficient for implementing the intrusion detection system:

Component	Specification
Processor	Intel Core i5 or above
RAM	8 GB
Storage	256 GB SSD
Operating System	Windows 10/11

3.12 Software Requirements

Software	Purpose
Python	Programming Language
Jupyter Notebook	Development Environment
Scikit-learn	Machine Learning
Pandas	Data Handling
NumPy	Numerical Computation
Matplotlib	Data Visualization
Seaborn	Statistical Visualization

4. IMPLEMENTATION

The implementation of the proposed Intrusion Detection System (IDS) was carried out using machine learning techniques to classify network traffic as either normal or malicious. The complete implementation process includes dataset preparation, preprocessing, feature engineering, model training, evaluation, and visualization of results. The system was developed using Python and machine learning libraries in a Jupyter Notebook environment.

4.1 Dataset Loading

The implementation started with loading the NSL-KDD formatted dataset into the system using the Pandas library. The dataset contains network traffic records with multiple features representing different network connection properties.

Steps Performed:

- Imported the dataset in CSV format.
- Verified dataset dimensions and feature count.
- Checked for missing or duplicate values.
- Examined attack labels and traffic categories.

Python Libraries Used:

- Pandas
- NumPy

The dataset consisted of:

- 22,544 records
- 41 input features
- 1 target label

The target variable represents:

- 0 → Normal Traffic
- 1 → Malicious Traffic

4.2 Data Preprocessing

Raw network traffic data cannot be directly used for machine learning because it contains categorical attributes and inconsistent numerical scales. Therefore, preprocessing was performed to prepare the data for model training.

4.2.1 Label Transformation

The original dataset contained multiple attack categories such as:

- DoS
- Probe
- R2L
- U2R

These attack categories were converted into a binary classification problem:

- Normal Traffic = 0

- Attack Traffic = 1

4.2.2 Categorical Feature Encoding

Several features such as:

- protocol_type
- service
- flag

were categorical in nature. Machine learning models require numerical input, so One-Hot Encoding was applied.

This converted each category into binary vectors.

4.2.3 Feature Scaling

Numerical attributes had varying ranges. To improve model performance and stability, Standard Scaling was applied.

Feature scaling ensured:

- Faster convergence
- Improved accuracy
- Better optimization

4.3 Dataset Splitting

The dataset was divided into:

- Training Data → 80%
- Testing Data → 20%

This allows the model to learn patterns from training data and evaluate performance on unseen data.

4.4 Machine Learning Model Training

Three supervised machine learning algorithms were implemented and evaluated.

4.4.1 Logistic Regression

Logistic Regression is a statistical classification algorithm used for binary classification problems.

Characteristics:

- Simple and fast
- Suitable for linear classification
- Performs well on structured datasets

4.4.2 Linear Support Vector Machine (SVM)

Support Vector Machine identifies the optimal hyperplane to separate classes.

Characteristics:

- Effective in high-dimensional spaces
- Good classification capability
- Reduces overfitting

4.4.3 Random Forest Classifier

Random Forest is an ensemble learning algorithm that combines multiple decision trees.

Characteristics:

- High accuracy
- Handles complex data efficiently
- Reduces variance and overfitting

The Random Forest classifier achieved the best performance among all models.

4.5 Intrusion Detection Process

Once training was completed, the system analyzed incoming network traffic and predicted whether it was normal or malicious.

Prediction Process:

1. Capture network traffic features.
2. Preprocess input data.
3. Apply trained machine learning model.
4. Generate prediction.
5. Display alert if malicious activity is detected.

4.6 Performance Evaluation

The performance of each machine learning model was evaluated using standard classification metrics.

Metrics Used:

- Accuracy
- Precision
- Recall
- F1-Score
- ROC-AUC

4.7 Confusion Matrix Generation

Confusion matrices were generated to analyze classification errors.

The matrix helps identify:

- True Positives
- True Negatives
- False Positives
- False Negatives

Random Forest produced the fewest misclassifications.

4.8 ROC Curve Visualization

Receiver Operating Characteristic (ROC) curves were generated to evaluate model discrimination capability.

The Random Forest model achieved:

- ROC-AUC Score ≈ 0.9997

which indicates excellent intrusion detection capability.

4.9 Feature Importance Analysis

Feature importance analysis was performed using the Random Forest classifier.

Important features identified:

- src_bytes
- dst_bytes
- flag

These features significantly contributed to identifying malicious traffic patterns.

5. Major Findings / Outcomes / Results

The proposed project, **"Machine Learning Enabled Intrusion Detection for Intelligent Network Security,"** successfully demonstrated the effectiveness of machine learning algorithms in identifying malicious network activities with high accuracy and reliability. The major findings and outcomes of the project are discussed below:

5.1 Successful Detection of Network Intrusions

The developed intrusion detection system was able to successfully classify network traffic into two categories:

- **Normal Traffic**
- **Malicious/Attack Traffic**

The system analyzed different network parameters and detected suspicious behavior automatically without human intervention. This demonstrates the practical capability of machine learning models in strengthening cybersecurity systems.

5.2 Performance Comparison of Machine Learning Models

Three supervised machine learning algorithms were implemented and evaluated:

- Logistic Regression
- Linear Support Vector Machine (SVM)
- Random Forest

The comparative analysis revealed significant differences in detection performance.

Model	Accuracy	Precision	Recall	F1-Score
Logistic Regression	94.34%	95.10%	93.53%	94.31%
Linear SVM	94.32%	95.33%	93.53%	94.27%
Random Forest	99.26%	99.24%	99.29%	99.27%

The **Random Forest classifier** achieved the best overall performance among all evaluated models.

5.3 Random Forest Produced Highest Accuracy

One of the major findings of the project is that the **Random Forest algorithm significantly outperformed** Logistic Regression and Linear SVM.

Key observations include:

- Accuracy of **99.26%**
- Very low false positive rate
- Better generalization capability
- Strong performance on complex network traffic patterns
- Reduced overfitting due to ensemble learning

This proves that ensemble-based algorithms are highly effective for intrusion detection systems.

5.4 Improvement in Detection Reliability

The system achieved high:

- **Precision**, meaning fewer false alarms
- **Recall**, meaning better attack detection capability
- **F1-Score**, showing balanced classification performance

This is important because intrusion detection systems must not only detect attacks accurately but also minimize unnecessary alerts that may overwhelm administrators.

5.5 Effective Data Preprocessing Improved Model Performance

Data preprocessing techniques contributed significantly to model efficiency and stability.

The following preprocessing methods were successfully applied:

- Label transformation
- One-Hot Encoding
- Feature normalization
- Dataset balancing and splitting

These preprocessing steps improved:

- Learning efficiency
- Classification accuracy
- Model stability
- Computational performance

5.6 Feature Importance Analysis

The project identified the most important network traffic features responsible for intrusion detection.

The Random Forest model highlighted features such as:

- src_bytes
- dst_bytes
- flag
- service
- protocol_type

These features played a major role in differentiating malicious traffic from normal network activity.

This finding helps improve:

- Network monitoring
- Attack pattern understanding
- Security analysis

5.7 ROC Curve and AUC Performance

The Receiver Operating Characteristic (ROC) analysis demonstrated excellent classification capability.

Key result:

- **AUC Score ≈ 0.9997**

This indicates:

- Near-perfect discrimination between attack and normal traffic
- Strong prediction confidence
- Reliable intrusion detection performance

5.8 Reduction in False Positives and False Negatives

A major challenge in intrusion detection systems is reducing:

- False Positives (normal traffic classified as attacks)
- False Negatives (attacks classified as normal)

The proposed Random Forest model achieved:

- Very few misclassifications
- Better threat detection
- Improved network security reliability

This makes the system suitable for real-time security monitoring environments.

5.9 Competitive Performance Compared to Existing Research

The proposed system achieved performance comparable to modern state-of-the-art intrusion detection approaches.

Compared with recent studies:

- The proposed model achieved competitive accuracy
- Required lower computational complexity
- Offered simpler implementation
- Provided faster training time

This demonstrates the practicality of the developed system for real-world deployment.

5.10 Real-Time Security Monitoring Capability

The system architecture supports continuous monitoring of network traffic.

Major operational outcomes include:

- Real-time packet analysis
- Continuous traffic monitoring
- Automatic intrusion prediction
- Alert generation for suspicious activities

This shows that the developed IDS can be extended into a real-time intelligent cybersecurity solution.

6. Conclusion and Future Scope

Conclusion

This project presented a machine learning based Intrusion Detection System (IDS) for intelligent network security. The system was developed using supervised machine learning algorithms including Logistic Regression, Linear SVM, and Random Forest. The NSL-KDD dataset was used for training and testing the models.

Among all the evaluated models, the Random Forest classifier achieved the best performance with the highest accuracy and minimum false predictions. The experimental results proved that machine learning techniques can effectively detect malicious network activities and improve cybersecurity systems.

The proposed system provides reliable intrusion detection, reduces manual monitoring efforts, and enhances overall network security performance.

Future Scope

The future scope of this project includes several improvements and advanced implementations:

- Integration of deep learning techniques such as CNN and LSTM for better detection accuracy.
- Development of a real-time intrusion detection system for live network monitoring.
- Deployment of the system in cloud and IoT environments for enhanced cybersecurity.
- Implementation of explainable AI techniques to improve model transparency and trust.
- Use of larger and more modern datasets to improve robustness against evolving cyber threats.
- Development of automated response mechanisms for intrusion prevention and threat mitigation.

These improvements can help build more intelligent, scalable, and efficient cybersecurity solutions in the future.

7.References

[1] Ahmad, "Network intrusion detection system: A systematic study of machine learning and deep learning approaches," *Transactions on Emerging Telecommunications Technologies*, vol. 32, e4150, 2021.

[2] Z. Yang, X. Liu, T. Li, D. Wu, J. Wang, Y. Zhao, and H. Han, "A systematic literature review of methods and datasets for anomaly-based network intrusion detection," *Computers & Security*, vol. 116, Art. 102675, 2022.

[3] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in *ICISSP 2018*, 2018.

[4] Canadian Institute for Cybersecurity, "Intrusion detection evaluation dataset (CIC-IDS2017)," dataset portal (includes labeled flows and CICFlowMeter-derived CSVs), accessed 2026.

[5] N. Moustafa and J. Slay, "UNSW-NB15: A comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set)," in *MilCIS 2015*, IEEE, 2015.

[6] University of New South Wales, "The UNSW-NB15 Dataset," project page describing dataset generation and attack categories, accessed 2026.

[7] M. Tavallaee, E. Bagheri, W. Lu, and A. A. Ghorbani, "A detailed analysis of the KDD CUP 99 data set," in *IEEE Symposium on Computational Intelligence for Security and Defense Applications (CISDA)*, 2009.
[8] Canadian Institute for Cybersecurity, "ISCX NSL-KDD dataset page" (notes benchmark role and limitations; dataset availability statement), accessed 2026.
[9] F. Ullah, S. Ullah, G. Srivastava, and J. C.-W. Lin, "IDS-INT: Intrusion detection system using transformer-based transfer learning for imbalanced network traffic," *Digital Communications and Networks*, vol. 10, no. 1, pp. 190–204, 2024.
[10] W. Xu and Y. Fan, "Intrusion Detection Systems Based on Logarithmic Autoencoder and XGBoost," *Security and Communication Networks*, 2022, Art. 9068724. (Publisher page was not directly accessible in this environment; citation based on indexed metadata.)
[11] S. T. Hamidou and A. Mehdi, "Enhancing IDS performance through a comparative analysis of Random Forest, XGBoost, and Deep Neural Networks," *Machine Learning with Applications*, vol. 22, Art. 100738, 2025.
[12] D. R. Patil, T. M. Pattewar, T. S. Shinde, K. S. Kumavat, and S. N. Deshpande, "Optimizing Network Intrusion Detection Systems Through Ensemble Learning and Feature Selection Using the CIC-IDS2017 Dataset," *Informatica*, vol. 49, pp. 641–670, 2025.
[13] R. Mohammad *et al*., "Enhancing Intrusion Detection Systems Using a Deep Learning and Data Augmentation Approach," *Systems*, 2024. (Architecture and dataset coverage documented in paper tables.)
[14] A. Zahoor, W. Abbasi, M. Z. Babar, and A. Aljohani, "Robust IoT security using isolation forest and one class SVM algorithms," *Scientific Reports*, vol. 15, Art. 36586, 2025.
[15] M. A. Abdel-Rahman, A. S. Alluhaidan, S. A. El-Rahman, A. E. Masnour, A. S. I. Amar, M. A. Sobh, A. M. Bahaa-Eldin, and T. Shamseldin, "Feature importance guided autoencoder for dimensionality reduction in intrusion detection systems," *Scientific Reports*, vol. 16, Art. 5013, 2026, doi:10.1038/s41598-026-36695-9.
[16] P. Waghmode *et al*., "Intrusion detection system based on machine learning…" (binary IDS evaluated on NSL-KDD, CICIDS2017, and UNSW-NB15 with multiple classifiers), 2025.
[17] D. Liu *et al*., "Deep Learning-Based Intrusion Detection: A CNN-LSTM …" (hybrid CNN–LSTM approach on CICIDS2017), *ACM* publication, 2025
[18] https://www.unb.ca/cic/datasets/nsl.html

8. Appendices

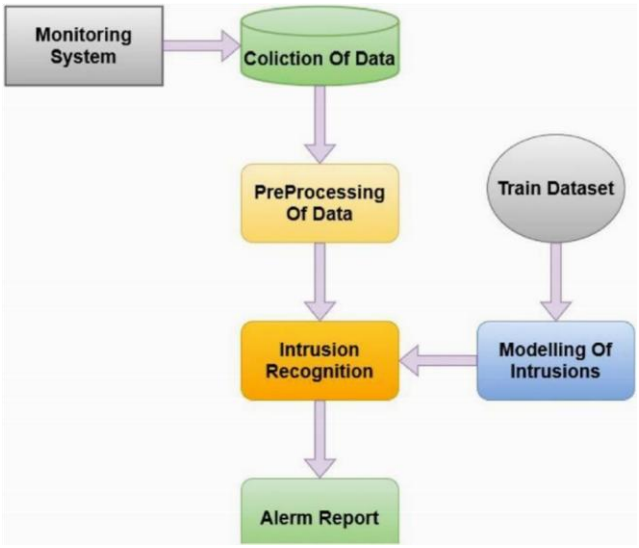

Fig.1. Real-time intrusion detection pipeline for monitoring network traffic and detecting malicious activities.

In this process the intrusion detection system will watch the network all the time. Find things that do not seem right. The intrusion detection system will make sure the network is safe, by watching the network activity and finding suspicious activity with the intrusion detection system.

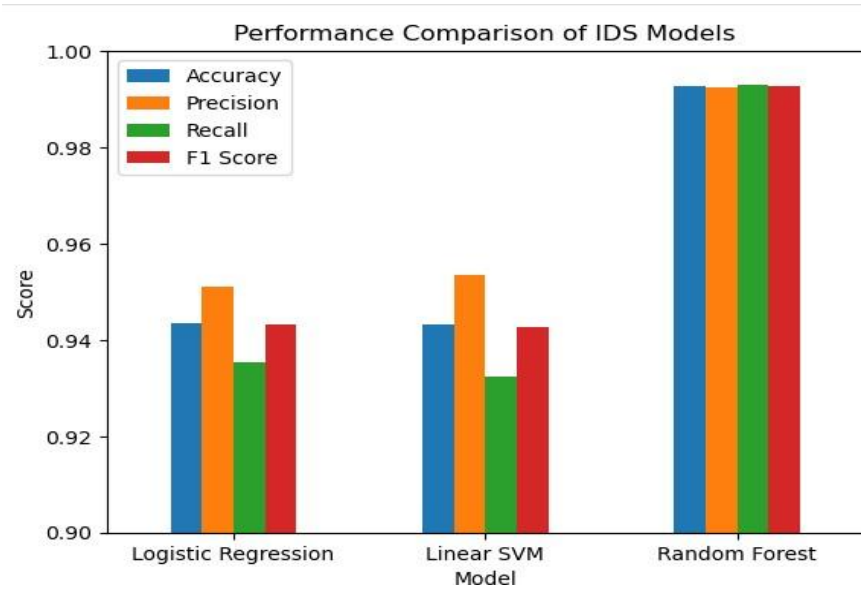

Fig.2. Performance comparison of Logistic Regression, Linear SVM, and Random Forest models.

The figure clearly shows that the **Random Forest model consistently outperforms the other classifiers across all evaluation metrics**, demonstrating its effectiveness in detecting malicious network activities.

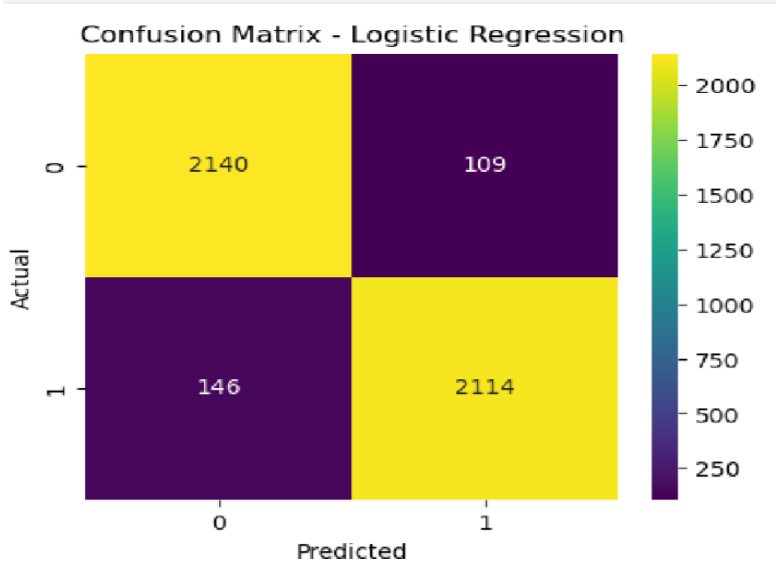

Fig.3. Confusion matrix for Logistic Regression intrusion detection model.

The Logistic Regression model correctly classified most normal and attack instances but produced a moderate number of false positives and false negatives.

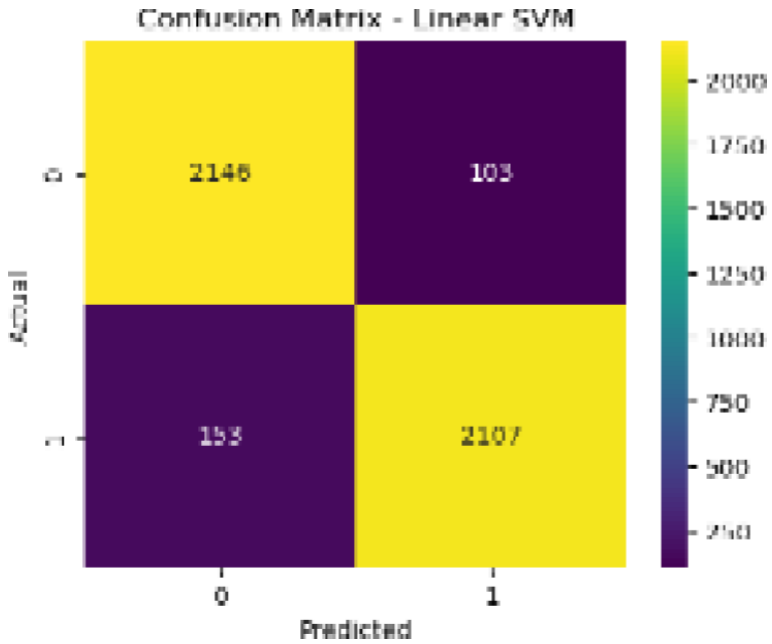

Fig.4. Confusion matrix for Linear SVM intrusion detection model.

The Linear SVM classifier showed similar performance to Logistic Regression but slightly improved precision by reducing some false positive predictions.

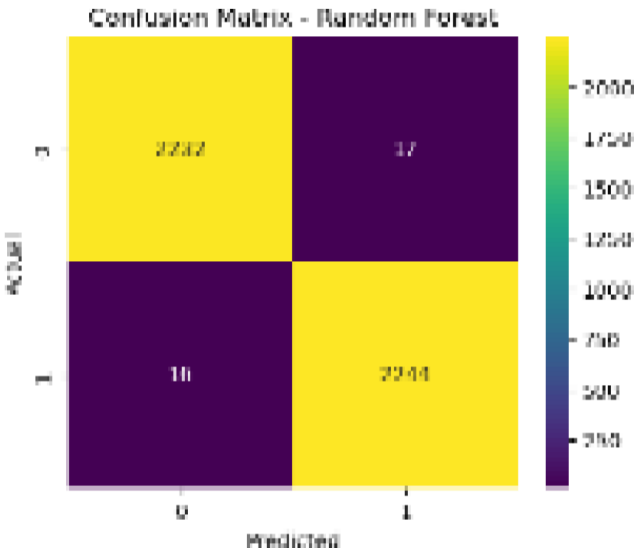

Fig. 5. Confusion Matrix for Random Forest intrusion detection model

The Random Forest classifier achieved the best classification performance with very few Misclassifications, demonstrating its strong capability in distinguishing normal and malicious network traffic

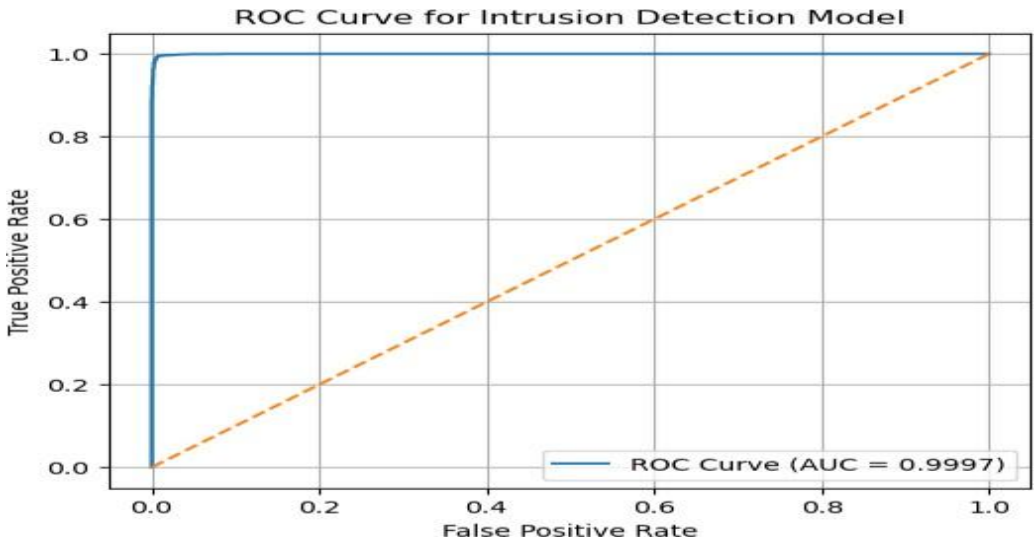

Fig. 6 Intrusion detection system ROC curve.

The ROC curve indicates the correlation between True Positive and False Positive rate (TPR and FPR). The model has an excellent discrimination capability with an AUC value of 0.9997 being close to 1.

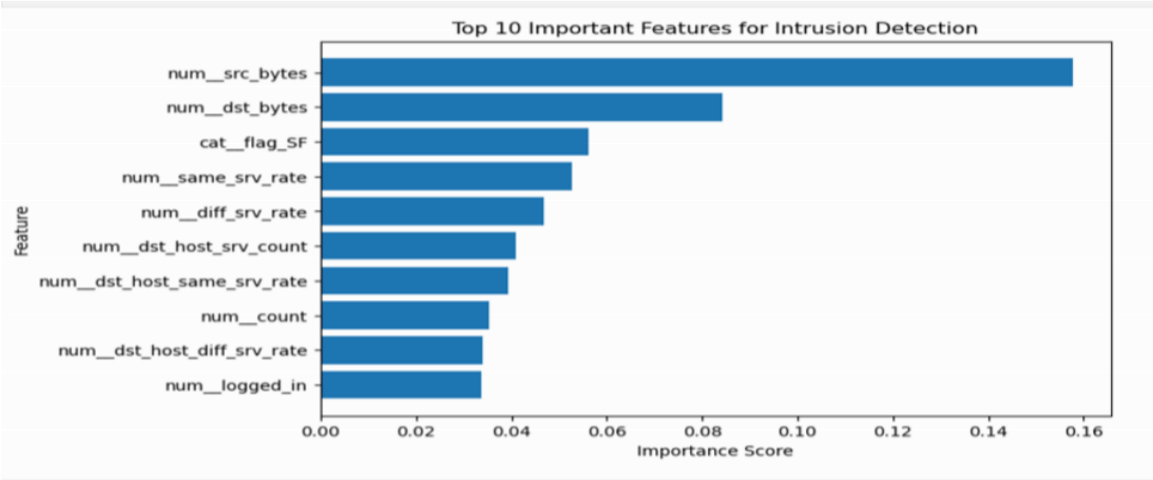

Fig. 7. The most significant features found by the classifier of the Random Forest.

The findings show that, the elements of src_byte, dst_byte and flag characteristics are important in differentiating between malicious and normal traffic. Knowing the importance of the features aids in enhancing the model interpretation and gives information about the nature of the network attacks.

Study	Dataset	Method	Accuracy
[1]	NSL-KDD	ML Survey	98.5%
[2]	Multiple	Anomaly Detection	98.8%
[11]	NSL-KDD	RF+SMOTE	99.80%
[12]	CIC-IDS2017	Ensemble Learning	99.10%
Proposed Model	NSL-KDD	Random Forest	99.26%

Table: Comparison with State-of-the-Art Methods